

□ UMDE Unified Architecture Strategy

Objective: Design a validation platform that is simultaneously **cost-efficient**, **high-performance**, and **scalable** — while meeting enterprise-grade standards expected by **FANG-class engineers** and supporting real-world health plan volumes.

□ Dual Mission

- ☒ **Cost-Optimized Mode**: Minimal infrastructure cost for everyday usage (10K–100K rows/day)
 - ☐ **High-Throughput Mode**: Accelerated batch processing (1M+ rows in under a minute)
 - ☐ Both are fully HIPAA, HITRUST, ISO 27001, and ISO 27701 compliant
-

□ Core Platform Foundations

- DAG-based rule dependency engine (declarative, traceable)
 - Plugin-based rule execution (modular, testable)
 - Redis cache for DAGs and config
 - Audit pipeline with S3 + TTL
 - Asynchronous and parallel-friendly by design
-

□ Cost-Optimized Mode

Designed for production workloads under 100K rows/day at <\$60/month

□ Key Components

- Per-record Lambda with batched 25-rule evaluation
- Redis-cached DAGs and rule configs
- Audit logs stored only for failures, compressed in S3
- Lifecycle policies on all storage buckets

□ Outcome

- ~50ms/record runtime
 - Up to 100K rows/day
 - \$30–60/month operating cost
-

⚡ High-Throughput Mode

Designed to validate millions of rows in minutes — with parallelism and vectorization

□ Enhancements

- Vectorized rule engine (Polars or DuckDB)
- Horizontal sharding across Fargate or K8s pods
- Precompiled DAGs
- Optional Rust/Numba rule kernels for hot paths
- Async, chunked audit ingestion

□ Outcome

- 1M rows in <60s
- 20x faster than per-row logic
- \$10–20 per million-row job

□ Novelty and Differentiation

☑ What Makes UMDE FANG-Class

Trait	Description
DAG-Driven Rule Graph	Dependency-aware, debuggable, self-traversing
Rule Plugins as Contracts	Abstracted and hot-swappable per use case
Dual-mode Execution	Switch between Lambda vs Batch seamlessly
Redis-Oriented Hot Paths	High-speed config access, no I/O bottlenecks
JSON-Based Audit Trails	Easy for devs, legal, compliance, and tracebacks
Secure-by-Design Infra	Zero-trust, encrypted, lifecycle-controlled

□ Target KPIs

Metric	Value	Mode
Per-record latency	< 100ms	Cost-Optimized
Per-batch latency	< 60 seconds / 1M rows	High-Throughput
Monthly operating cost	<\$60	Cost-Optimized
Max batch size	5M+ rows	High-Throughput
Rule count supported	25+ (scalable to 100+)	Both

□ Strategic Recommendations

□ Refine Cost Model

- Adjust baseline target to \$50–60/month (realistic for 100K rows/day)
- Achieve \$8–11 savings via reserved DynamoDB/Redis
- For \$30–40/month tiers, explore moving non-critical metadata to S3 (pending compliance approval) or use Aurora Serverless for lower storage costs
- Publish per-rule cost benchmarks (e.g., \$0.0000005/rule execution) to earn trust with technical reviewers

⚡ Enhance High-Throughput Mode

- Replace Fargate with AWS Batch + Spot Instances for 1M+ row jobs
- Coordinate shards via DynamoDB streams or queue groups
- Cap memory per pod to 4 GB for predictability
- Benchmark Polars, DuckDB, and Spark to verify 20x claim

□ Improve Observability

- Integrate AWS X-Ray for Lambda/Fargate rule traceability
- Expose metrics for:
 - Rule latency
 - Cache evictions
 - Audit pipeline delivery time
- Set up anomaly detection (e.g., CloudWatch Alarms on error spikes)

□ □ Mitigate Cold Starts

- Extend provisioned concurrency to backend validation paths (not just UI flows)
- Add scheduled warm-up Lambda (daily ping; ~\$0.01/month)
- Benchmark impact of cold starts on 100K/day use case

□ Reduce Vendor Lock-In

- Abstract SQS (e.g., via SNS abstraction)
- Decouple DynamoDB with OpenSearch fallback for multi-cloud support
- Use Terraform for consistent provisioning
- Document equivalent GCP/Azure mappings (Pub/Sub, Firestore, Blob)

□ Strengthen Testing

- Add chaos tests for Redis eviction, Lambda timeout, DAG corruption
 - Simulate 5M row batch with 10% malformed records
 - Publish rule success/failure rates (target 99.9%) to meet FANG-style quality scrutiny
-

□ SOW Model & Engagement Process

Every customer engagement begins with a risk-free pilot (typically 1 month), leveraging UMDE-managed infrastructure and Safe Harbor-compliant or synthetic data files. The pilot phase includes AI-assisted rule extraction, human-in-the-loop validation review, field-level reconciliation, and end-to-end audit logging. Upon meeting mutually agreed success metrics, customers transition to a 12-month production contract under a formal Statement of Work (SOW) linked to the cost and performance targets outlined above.

Key Elements:

- Pilot scope, objectives, and data standards
- Performance and cost targets (referenced from KPIs)
- Process for transition to production SOW
- Ongoing benchmarks and volume commitments

□ Conclusion

UMDE is architected to: - Serve small-to-mid health plans affordably - Scale up for data-rich enterprise clients - Meet the expectations of technical reviewers at the top of the stack - And do all of this with auditable, modern, compliant infrastructure

This isn't just scalable — it's impressive. And it's ready.